

Reporte Proyecto

Alumno: Luis Hugo Samuel Gutierrez Luna

Docente: Ricardo Alfredo Monroy Rodríguez

Este código implementa una aplicación de consola en Python para la gestión y optimización de producción de calzado Nike. Su objetivo principal es calcular la distribución óptima de pares por talla basándose en la demanda estimada para evitar el sobreinventario en outlets, gestionar archivos de texto de soporte y simular sesiones de usuario.

Funcionalidades principales por módulo

1. Autenticación e Inicio de Sesión (`pantalla_carga`, `iniciar_sesion`, `solicitar_fecha`)

Muestra una animación simple simulando la carga del sistema.

Solicita el nombre del usuario y genera un banner de bienvenida.

Pide la fecha de operación validando que esté en formato correcto (`dd/mm/aaaa`) antes de almacenarla como una tupla (día, mes, año).

2. Lectura y Gestión de Archivos (`obtener_modelos_catalogo`, `cargar_porcentajes_demanda`, `listar_y_leer_archivos`)

Obtención del catálogo: Intenta leer los modelos desde `catalogo_nike.txt`; si no existe, usa una lista por defecto con 12 modelos predefinidos.

Proyección de demanda: Lee `proyeccion_demandas.txt` para obtener los porcentajes de demanda por talla. Si falla, asigna un diccionario con valores estándar por defecto.

Lector de archivos: Permite consultar visualmente desde la consola el contenido de archivos del sistema como `catalogo_nike.txt`, `ventas_historicas.txt`, `proyeccion_demandas.txt` o `politica_outlets.txt`.

3. Cálculo de Producción y Generación de Reportes (`registrar_ajuste_produccion`)

Despliega la lista con los modelos de tenis autorizados y obliga al usuario a elegir una opción válida por número o por nombre exacto.

Solicita la cantidad total de pares a fabricar y valida que sea un número mayor a cero.

Distribuye las unidades totales entre las distintas tallas según los porcentajes de demanda cargados.

Crea automáticamente la carpeta reportes/ (si no existe) y guarda/anexa el desglose en un archivo .txt nombrado según el modelo seleccionado.

4. Monitoreo de Sesión (verificar_inactividad_simulada)

Simula un temporizador de inactividad de 10 minutos mediante un bucle for. Si el usuario decide no continuar, la sesión se reinicia solicitando nuevamente usuario y fecha.

5. Menú Interactivo (desplegar_menu_matriz, main)

Presenta un menú estructurado con 5 opciones que controla el flujo general del programa mediante un bucle while.

Código

```
import datetime
import time
import os
```

```
def pantalla_carga():
    """Simula la carga inicial del sistema Nike (máximo 5 segundos)."""
    print("\n" + "=" * 60)
    print(" INICIANDO SISTEMA DE DISTRIBUCIÓN Y OPTIMIZACION NIKE... ")
    print("=" * 60)
    for i in range(1, 4):
        print(f"Cargando modulos de inventario y matriz DTC... [{i * 33}%]")
        time.sleep(1)
    print("¡Sistema listo para operar!\n")

def iniciar_sesion():
    """Solicita el nombre del usuario y genera un mensaje dinamico concatenado."""
    pantalla_carga()
    nombre = input("Por favor, ingresa tu nombre o nickname para acceder: ").strip()
    mensaje_bienvenida = ">>> Bienvenido al Sistema de Control de Producción Nike, " + nombre.upper() + " <<<"
    print("\n" + "*" * len(mensaje_bienvenida))
    print(mensaje_bienvenida)
    print("*" * len(mensaje_bienvenida) + "\n")
    return nombre

def solicitar_fecha():
```

```

"""Solicita la fecha y la almacena en una tupla estructurada: (dia, mes, año)."""
while True:
    try:
        fecha_str = input("Ingresa la fecha de operación en formato (dd/mm/aaaa):
").strip()
        partes = fecha_str.split("/")
        if len(partes) != 3:
            raise ValueError("El formato debe ser dd/mm/aaaa")

        dia = int(partes[0])
        mes = int(partes[1])
        año = int(partes[2])

        fecha_objeto = datetime.date(año, mes, dia)
        fecha = (dia, mes, año)
        print(f"-> Fecha registrada correctamente: {fecha[0]}/{fecha[1]}/{fecha[2]}\n")
        return fecha
    except ValueError as e:
        print(f"Error, ingresaste una fecha invalida: {e}. Intenta de nuevo.\n")

def cargar_porcentajes_demanda():
    """Lee el archivo proyeccion_demandas.txt para obtener las tallas y porcentajes
reales."""
    porcentajes = {}
    try:
        with open("proyeccion_demandas.txt", "r", encoding="utf-8") as archivo:
            next(archivo)
            for linea in archivo:
                linea = linea.strip()
                if linea:
                    talla, pct = linea.split(",")
                    porcentajes[talla.strip()] = float(pct.strip())
    except Exception:
        porcentajes = {"24": 0.05, "25": 0.10, "26": 0.20, "27": 0.30, "28": 0.20, "29":
0.10, "30": 0.05}
    return porcentajes

def obtener_modelos_catalogo():
    """Lee catalogo_nike.txt o retorna la lista fija de los 12 modelos exactos del
catálogo."""
    modelos = []
    if os.path.exists("catalogo_nike.txt"):
        try:
            with open("catalogo_nike.txt", "r", encoding="utf-8") as archivo:

```

```

        lineas = [l.strip() for l in archivo if l.strip()]
        if lineas and ("modelo" in lineas[0].lower()):
            lineas = lineas[1:]
        for linea in lineas:
            partes = linea.split(",")
            if partes:
                modelos.append(partes[0].strip())
    except Exception:
        pass
if not modelos:
    modelos = [
        "Air Force 1", "Air Max 90", "Pegasus 40", "Dunk Low",
        "Air Jordan 1 Low", "Air Jordan 1 mid", "Air Jordan 1 high",
        "Air Jordan 3", "Air Jordan 4", "Air Jordan 5",
        "Air Jordan 6", "Jordan Spizike Low"
    ]
return modelos

```

def listar_y_leer_archivos():

"""Muestra los 4+ archivos .txt en forma de diccionario y permite leer su contenido."""

```

archivos_disponibles = {
    "1": "catalogo_nike.txt",
    "2": "ventas_historicas.txt",
    "3": "proyeccion_demandas.txt",
    "4": "politica_outlets.txt"
}

```

```

print("\n--- ARCHIVOS DE SISTEMA DISPONIBLES EN DISCO ---")

```

```

for clave, nombre_archivo in archivos_disponibles.items():
    print(f" [{clave}] -> {nombre_archivo}")

```

opcion = input("\nSelecciona el numero o escribe el nombre del archivo a consultar: ").strip()

```

nombre_archivo = archivos_disponibles.get(opcion, opcion)

```

```

try:

```

```

    with open(nombre_archivo, "r", encoding="utf-8") as archivo:
        print(f"\n===== CONTENIDO DE: {nombre_archivo} =====")
        for linea in archivo:
            print(linea.strip())
        print("=" * 45 + "\n")

```

```

except FileNotFoundError:

```

```

    print(f"\n[ERROR]: El archivo '{nombre_archivo}' no existe en el directorio.")

```

```

except PermissionError:
    print(f"\n[ERROR]: No tienes permisos de lectura para el archivo
'{nombre_archivo}'.")
except Exception as e:
    print(f"\n[ERROR INESPERADO]: {e}")

def registrar_ajuste_produccion(fecha_tupla):
    """Calcula la distribución por tallas y guarda el reporte en un nuevo archivo .txt."""
    print("\n--- CÁLCULO DE PRODUCCIÓN OPTIMIZADA PARA EVITAR OUTLETS
---")

    modelos_disponibles = obtener_modelos_catalogo()
    print("\nModelos autorizados en el catálogo:")
    for idx, mod in enumerate(modelos_disponibles, start=1):
        print(f" [{idx}] {mod}")

    modelo_seleccionado = None
    while True:
        opcion_m = input("\nSelecciona el número (1-12) o escribe el nombre exacto
del modelo: ").strip()

        if opcion_m.isdigit():
            num = int(opcion_m)
            if 1 <= num <= len(modelos_disponibles):
                modelo_seleccionado = modelos_disponibles[num - 1]
                break

        for mod in modelos_disponibles:
            if mod.lower() == opcion_m.lower():
                modelo_seleccionado = mod
                break

        if modelo_seleccionado:
            break

    print("\n[ERROR]: Modelo no válido. Solo puedes seleccionar uno de los 12
modelos del catálogo.")

    while True:
        try:
            total_unidades = float(input(f"\nIngresa la cantidad total de unidades a
producir para '{modelo_seleccionado}': "))
            if total_unidades > 0:

```

```

        break
    print("[ERROR]: La cantidad total debe ser mayor a cero.")
except ValueError:
    print("[ERROR DE ENTRADA]: Debe ingresar un valor numérico válido.")

tallas_porcentajes = cargar_porcentajes_demanda()

print(f"\nProcesando distribucion optimizada para: {modelo_seleccionado}")
print("Talla\t\tPorcentaje\t\tUnidades Asignadas")
print("-" * 55)

lineas_reporte = []
total_distribuido = 0

for talla, pct in tallas_porcentajes.items():
    unidades_talla = round(total_unidades * pct)
    total_distribuido += unidades_talla
    print(f"Talla {talla}\t\t{int(pct * 100)}%\t\t{unidades_talla} pares")
    lineas_reporte.append(f"Talla {talla}: {unidades_talla} pares ({int(pct * 100)}%)\n")

dia, mes, año = fecha_tupla
carpeta_reportes = "reportes"
if not os.path.exists(carpeta_reportes):
    os.makedirs(carpeta_reportes)

nombre_archivo = f"reporte_produccion_{modelo_seleccionado.lower().replace(' ', '_')}.txt"
ruta_reporte = os.path.join(carpeta_reportes, nombre_archivo)

try:

    with open(ruta_reporte, "a", encoding="utf-8") as f:
        f.write("\n===== \n")
        f.write(f"REPORTE NIKE - FECHA DE EMISION: {dia}/{mes}/{año}\n")
        f.write(f"Modelo: {modelo_seleccionado}\n")
        f.write(f"Produccion Objetivo Total: {int(total_unidades)} pares\n")
        f.write(f"Total Distribuido Optimizado: {int(total_distribuido)} pares\n")
        f.write("--- Desglose por Tallas ---\n")
        for l in lineas_reporte:
            f.write(l)
        f.write("Estatus: Distribucion verificada para evitar sobreinventario en Outlets.\n")
        f.write("===== \n")

```

```
        print(f"\n[ÉXITO]: Reporte guardado y anexado exitosamente en  
'{ruta_reporte}'.")
```

```
except ValueError:
```

```
    print("[ERROR DE ENTRADA]: Debe ingresar un valor numerico valido.")
```

```
except Exception as e:
```

```
    print(f"[ERROR AL ESCRIBIR ARCHIVO]: {e}")
```

```
def desplegar_menu_matriz():
```

```
    """Despliega las opciones en formato de Matriz (Filas x Columnas)."""
```

```
    matriz_menu = [
```

```
        ["1. Leer Archivo .TXT", "2. Calcular Distribucion"],
```

```
        ["3. Simular Inactividad", "4. Modificar Usuario  "],
```

```
        ["5. Salir del Sistema ", "          "]
```

```
    ]
```

```
print("\n+=====+")
```

```
    print("|          MENU PRINCIPAL          |")
```

```
    print("+=====+")
```

```
    for fila in matriz_menu:
```

```
        print(f"| {fila[0]} | {fila[1]} |")
```

```
    print("+=====+")
```

```
def verificar_inactividad_simulada():
```

```
    """
```

```
    Cumple el Requerimiento 5 utilizando un ciclo for para medir  
    el periodo de tiempo sin pausar el menú al inicio de cada acción.
```

```
    """
```

```
    print("\n[Simulando monitoreo de inactividad de 10 minutos con ciclo FOR...]")
```

```
    for minuto in range(1, 11):
```

```
        time.sleep(0.05)
```

```
    print("\n" + "!" * 60)
```

```
    print("AVISO: Han transcurrido 10 minutos de inactividad.")
```

```
    respuesta = input("¿Desea continuar en el menú del sistema? (escribe 'si' o 'no'):  
").strip().lower()
```

```
    print("!" * 60)
```

```
    return respuesta == "si"
```

```
def main():
```

```
    usuario_actual = iniciar_sesion()
```

```

fecha_operacion = solicitar_fecha()

ejecutando = True
while ejecutando:
    desplegar_menu_matriz()
    opcion = input("Selecciona una opción del menu (1-5): ").strip()

    if opcion == "1":
        listar_y_leer_archivos()
    elif opcion == "2":
        registrar_ajuste_produccion(fecha_operacion)
    elif opcion == "3":
        continuar = verificar_inactividad_simulada()
        if not continuar:
            print("\n[SESIÓN SUSPENDIDA]: Regresando a la pantalla de inicio...")
            usuario_actual = iniciar_sesion()
            fecha_operacion = solicitar_fecha()

    elif opcion == "4":
        print("\n--- CAMBIO DE USUARIO ---")
        usuario_actual = iniciar_sesion()
        fecha_operacion = solicitar_fecha()
    elif opcion == "5":
        print(f"\nGracias por utilizar el Sistema de Distribucion Nike, {usuario_actual}.
¡Hasta luego!")
        ejecutando = False
    else:
        print("\n[OPCIÓN INVÁLIDA]: Por favor selecciona un número entre 1 y 5.")

if __name__ == "__main__":
    main()

```